formance. Second, it avoids both the in-memory index and buffered disk writes during merge, thus saving memory. Third, it is possible for direct insert to be as efficient as buffered insert, as inserts can not be efficiently batched.

However, it is non-trivial to do direct insert efficiently. There are several challenges as follows.

Challenge 1. *Each insert updates tens or hundreds of neighbor records. Applying the updates in place causes massive random SSD writes, while using a log-structured layout suffers from frequent garbage collection.*

In a graph-based index, each vector connects to tens or hundreds of vectors for navigation [11, 27]. According to Algorithm 2, bi-directional edges are added to the target vector and its neighbors during insert. This process updates the neighbor records scattered randomly on tens or hundreds of pages [27], which leads to massive random SSD writes.

For fewer writes, a straightforward solution is to use a log-structured data layout. However, inserting 10% new records makes the index 12.8× larger (assuming an out-degree of 128); only 7.9% of the records are valid. Thus, garbage collection, which gathers the valid records into a new index, is frequently required. This slows down frontend search requests.

Challenge 2. *Ensuring the isolation of direct insert and search harms their concurrency, due to near-root lock contention. However, the potential of using the approximate index feature for better concurrency is under-exploited.*

According to Algorithm 2, insert may update all the records explored in the search path. To ensure the isolation of insert and search, a naive approach is to lock all of the involved records before updating. However, the record set contains near-root records. Locking them causes serious lock contention, blocking concurrent inserts and searches.

An alternative approach is to leverage the approximate feature of operations for fine-grained concurrency control: Atomicity and isolation of operations are not mandatory for an approximate index, so they can be relaxed for better concurrency. However, how to design an efficient concurrency control protocol for on-disk graph-based indexes, which efficiently utilizes this feature for shorter critical sections and better concurrency, is still under-exploited.

3 Design and Implementation

We design OdinANN, a billion-scale graph-based ANNS index supporting direct inserts and buffered deletes. OdinANN achieves efficient direct insert with the following goals.

- **Reduced disk writes.** Direct insert in a graph index updates tens or hundreds of neighbor records. We do space overprovision to combine these updates into fewer disk writes, while not suffering from GC overhead (§3.2).

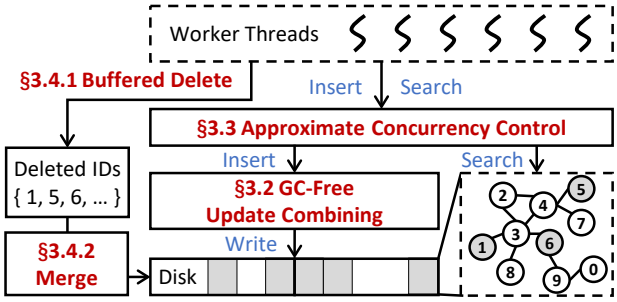

Figure 3: OdinANN overview.

- **High concurrency.** We leverage the approximate feature of operations to relax the isolation level of both insert and search for better concurrency (§3.3).

3.1 Overview

Figure 3 shows the overview of OdinANN.

Graph layout. The directed graph in OdinANN is organized as adjacent lists on disk (Figure 1), similar to previous works [24, 27]. Differently, OdinANN does space overprovision on disk for multiple free records on a page. Out-of-place record updates are combined in these pages (§3.2). In memory, PQ-compressed vectors are stored for graph navigation.

Operations. OdinANN supports three types of operations, vector search, insert, and delete.

- *Vector search.* OdinANN follows Algorithm 1 for vector search. Differently, OdinANN adopts a dynamic candidate pool, whose size increases with the number of deleted vectors in the pool, to ensure search quality in workloads containing deletes (§3.4.1). For better concurrency, OdinANN only ensures per-record consistency instead of the atomicity of the whole search (§3.3).
- *Direct insert.* OdinANN follows Algorithm 2 for direct vector insert. It updates the records out of place in the overprovisioned disk space, to combine them for less write amplification (§3.2). For concurrency control, OdinANN uses an approximate snapshot of the target vector's neighbors during insert to reduce its critical section (§3.3).
- *Buffered delete.* OdinANN records the ID of deleted vectors in memory (§3.4.1) and merges delete to disk periodically (§3.4.2), similar to previous works [23, 30]. It has low runtime memory consumption (only for IDs) and little interference with frontend search tasks during the merge.

3.2 GC-Free Update Combining

In-place record updates suffer from massive disk writes, while the log-structured layout induces frequent garbage collection.

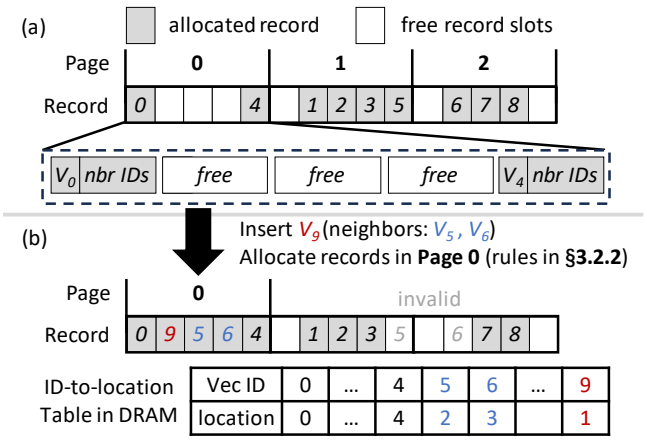

Figure 4: (a) OdinANN does space overprovision for more free records in a page. (b) Insert a vector and update its neighbors. Multiple record updates are combined into a single page.

To solve this issue, we do space overprovision to merge multiple record updates into a single page.

3.2.1 Out-of-place Record Updates for Combining

We find that fixed-size records of the graph (Figure 1) enable out-of-place record updates to be GC-free: If a record is updated out-of-place, its original place can be directly reused by another update, without explicit garbage collection.

Based on this idea, we do space overprovision to combine multiple record updates within a single page write. Specifically, OdinANN reserves extra record slots on disk, creating multiple free slots per page. During an insert, instead of modifying records in their original locations, OdinANN writes the updated records to these free slots. Our allocation strategy (§3.2.2) prioritizes allocating slots on the same page, thus achieving the combination of multiple out-of-place updates into one page. The original record locations are then marked as available for subsequent inserts.

Figure 4 illustrates this process. In Figure 4(a), 9 allocated records normally require two pages; however, OdinANN provisions three, leaving three free slots on page 0. This allows for up to three record updates to be combined. When inserting V_9 (Figure 4(b)), OdinANN updates V_9 and its neighbors, V_5 and V_6. These three record updates are combined into a single write to page 0. Then, the in-memory ID-to-location table is updated, and the original slots for V_5 and V_6 are marked for reuse in future inserts.

3.2.2 Record Allocation Rules

To maximize update combining, a straightforward way is to greedily allocate new records within pages with the most free slots. However, this approach requires read-modify-write for updating partially-filled pages, where the inherent write-read ordering leads to high latency.

We observe that vector insertion involves reading the pages containing the target vector's candidate neighbors. Caching these pages in memory allows us to update them without read-modify-write operations. Based on this observation, we design three record allocation rules, listed in order of priority:

Rule #1: empty rule. We first allocate records in empty pages where all the records are invalid, as using them does not require extra reads. Specifically, an empty page queue is stored in memory, which is updated when recycling records. Pages in the queue are used first during allocations.

Rule #2: on-path partial-empty rule. We allocate partial-empty pages only on the search path of insert, to avoid extra reads in updating them. Recall that each insert first searches the target vector for its candidate neighbors. The accessed pages are cached and directly used by neighbor updating. In this rule, only pages with less than m non-empty records are allocated, where m is a configurable parameter.

Rule #3: overprovision rule. If the two rules above can not allocate enough space for all the updated records, we allocate new pages on disk. Although this rule causes space overprovision, the overall space consumption is limited by Rules #1 and #2. We analyze it as follows.

3.2.3 Space Consumption and Write Amplification

Each page contains m non-empty records on average, resulting in $(n/m)\times$ space consumption compared to in-place record update, where n is #records per page. Compared to the log-structured layout, where n record updates are combined in a single page write, OdinANN combines $(n-m)$ record updates in a single page write. This concludes a write amplification of $n/(n-m)\times$. By default, OdinANN sets m to $\lfloor n/2 \rfloor$, so the space consumption and write amplification are $2\times$.

This analysis is based on the assumption that Rule #2 uses most pages with $< m$ non-empty records. In §4.5, we will show that this assumption holds in real-world datasets.

We find that such overprovisioning is cost-effective. In our billion-scale evaluation (§4.3), OdinANN uses $2\times$ disk space (~1TB extra) but ~128GB less peak memory, compared to DiskANN (buffered insert). The price trend [20] shows that a 1TB SSD (~$100) is cheaper than 128GB DRAM (>$200).

3.3 Approximate Concurrency Control

Search and insert are both *approximate* processes. Search returns an approximate top-k of the target vector, and insert selects an approximate neighbor set for the target vector using a top-k search. These operations are not required to be atomic and isolated, to achieve better concurrency.

We leverage such approximate features to design an efficient concurrency control protocol for search and insert. Specifically, searches see a consistent snapshot for *each*

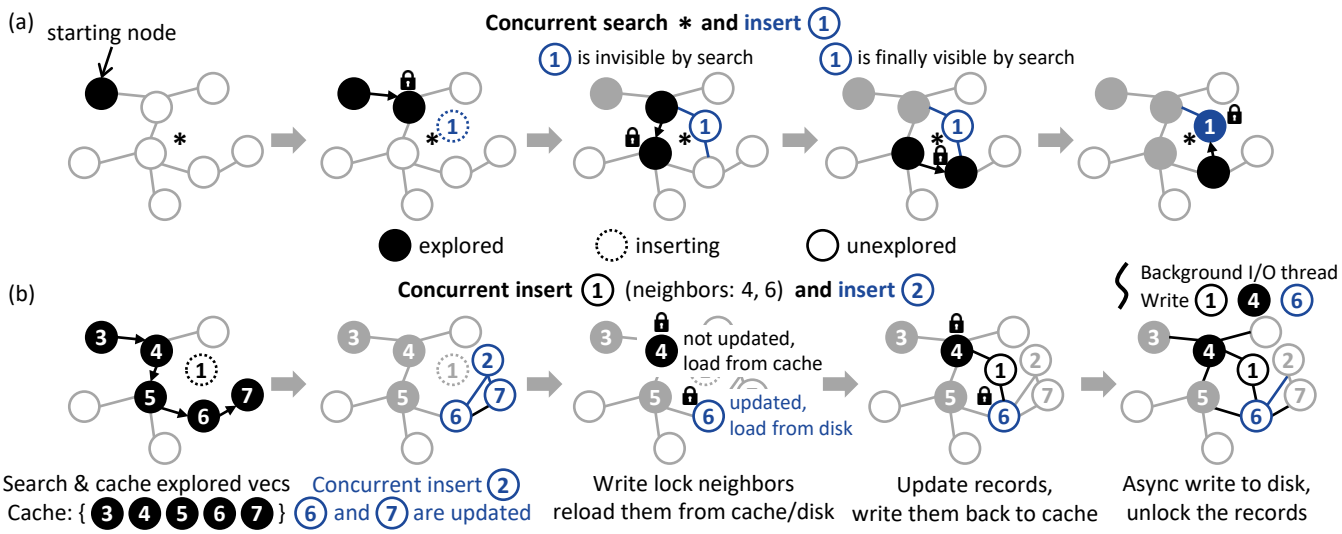

Figure 5: Approximate concurrency control. (a) Search sees a consistent snapshot for each record instead of the whole graph (e.g., record ① can be invisible before Step 4 but visible after it). (b) Inserts link the target vector with an *approximate snapshot* of neighbors. The snapshot may not be the best neighbor set (e.g., ② is a better neighbor of ① than ⑥).

record instead of the whole graph, while inserts see *approximate snapshots* for their candidate neighbor sets.

Search-insert conflicts. The search thread only holds locks when it reads a record (line 8 in Algorithm 1) to avoid getting a torn record. As out-of-place record update is adopted, locking only the ID-to-location mapping item ensures a consistent record snapshot, with no need to lock the page. In other periods, no locks are held for better concurrency.

Figure 5(a) shows an example. The search process includes 5 steps. In each step, only the per-record ID-to-location lock is held for the target record. In Step 3, a new record ① is inserted, with updates to the records on the search path. As we do not ensure a consistent snapshot for the whole graph (i.e., atomic search operation), we allow ① to be invisible by the search process before Step 4 and visible after it.

Insert-insert conflicts. Insert follows the process of Algorithm 2. Figure 5(b) shows an example of concurrent inserts, where vector ① is inserted by executing Steps 1, 3, 4, and 5, and vector ② is concurrently inserted in Step 2.

To insert a vector ①, OdinANN first searches ① to get the explored vectors in the search path, which are then pruned to generate ①'s candidate neighbors (④ and ⑥, Step 1). The acquired neighbor set is not accurate but an *approximate snapshot*, since new records may be concurrently inserted (e.g., ② in Step 2). Second, it acquires the per-record write locks of the involved records, as well as the per-page write locks for pages to be updated (Step 3). Third, it validates the neighbor records by reloading them, to avoid update loss (e.g., loss of ⑥'s update, Step 3). Fourth, it updates the involved records (①, ④, and ⑥) on disk, using the approach in Section 3.2 (Step 4). Finally, it releases all the locks (Step 5).

This process seems to be similar to optimistic concurrency control (OCC). However, OCC aims to achieve atomicity in operations. It searches the index again for validation and repeats the read-lock-validate process until the result set does not change. Instead, we directly view the obtained vector set as an approximate snapshot of candidate neighbors, thus avoiding the long latency of extra search.

We observe that this method is still not efficient enough, because of costly disk I/O and neighbor pruning in the critical section. We further propose two optimizations for them.

Optimization #1: move disk I/O out of critical sections. We find that the approximate neighbor snapshot enables using a write-back page cache to avoid disk I/O in the critical section. An insert contains one batched disk read (for record reload) and one batched disk write (for record updates) in the critical section. For the read, we admit the explored vectors to the cache (Step 1), thus reloading most of them from the cache (Step 3). For the write, we only write the record back to the cache (Step 4). To limit the cache space, cached pages are immediately freed after a search. A background I/O thread is used to commit the cached writes to disk (Step 5).

Optimization #2: delta neighbor pruning. Approximate concurrency control avoids most disk I/O in the critical section, by using Optimization #1. However, pruning (line 9 in Algorithm 2) becomes the bottleneck in the critical section. In DiskANN, pruning compares all the neighbor pairs to validate triangle inequality, which results in a complexity of $O(R^2)$, where R is the maximum out-degree. Billion-scale datasets require a large R (e.g., 128) for search performance, resulting in high pruning overhead.

We reduce the complexity of most neighbor pruning in insert from $O(R^2)$ to $O(R)$. This is inspired by an observation that the original neighbors mostly follow the triangular inequality. Therefore, instead of checking all the neighbor pairs, we only check the newly inserted neighbor with all the other neighbors. The farthest neighbor that does not follow triangular inequality is pruned.

If no neighbor can be pruned, we fall back to the original $O(R^2)$ pruning algorithm. We prune the first neighbor that does not follow triangular inequality, for early stopping while not breaking the characteristics of the original graph.

3.4 Buffered Delete and Merge

In §3.2 and §3.3, we show how OdinANN supports efficient direct inserts. In this section, we discuss how OdinANN supports buffered deletes and merging at a low cost. We choose buffered deletes instead of direct ones because they show little interference with frontend search during the merge (§4.7).

3.4.1 Buffered Delete

Overview. OdinANN records the IDs of deleted vectors in memory. When deleting a vector, OdinANN adds its ID to an in-memory deletion set, resulting in a low runtime memory footprint (4B or 8B per vector). Deleted vectors are excluded from the search results after the on-disk ANNS.

For concurrency control with search and insert, a global read-write lock is used. Delete operations acquire the write lock to update the delete set. After a search finishes on the disk, it acquires the read lock and excludes the deleted records from the search result. We do not use more fine-grained locks as delete has much less overhead than search and insert.

Challenge: search quality insurance. In workloads containing deletes, the on-disk search quality may be degraded by deleted vectors. Specifically, the results returned by the on-disk ANNS may contain deleted vectors, which are excluded from the results after the on-disk ANNS finishes. This process may lead to fewer than expected top-k results, even if we return more than k results in the on-disk search. In OdinANN, we should solve this issue without the assistance of the in-memory index, because it is avoided by direct insert.

Solution: dynamic candidate pool. As deleted nodes are excluded from the search result, there may be insufficient results even with a large candidate pool length l. To tackle the problem, we adapt the candidate pool size dynamically.

We find that the deleted vectors should only serve as indexing, but not contribute to the results. Motivated by this, we "ignore" the existence of deleted vectors in the candidate pool, namely, we let the candidate pool contain at most l vectors that are *not deleted*. To achieve it, we adjust the candidate pool size l after admitting the neighbors of the current exploring vector (line 12 in Algorithm 1).

3.4.2 Two-Pass Merge for Delete

When to merge. We use two metrics to decide when to merge. The first is the ratio of deleted vectors in the index. When it reaches a threshold (e.g., 10%), merging is triggered. This metric is straightforward but sometimes inaccurate. This is because vectors located in different places are not equally important. For example, near-root vectors influence every search request, while near-boundary vectors may only influence a small part. However, they are treated equally in the metric of deleted vector ratio.

We use the search I/O amplification as another metric. It is based on our observation that the disk I/O in the search process reflects the index quality. Given the same candidate pool size l, worse index quality results in a larger candidate pool, thus resulting in more disk I/O. Therefore, we record the average disk I/O during the search. Merging is triggered when the disk I/O is significantly more (e.g., 1.1×) than when no vectors are deleted.

How to merge. Merging delete in OdinANN includes two passes, in each pass OdinANN scans the whole index sequentially. This process is not as I/O intensive as merging inserts, so it shows little interference with frontend search (§4.7).

In the first pass, OdinANN scans the whole index to load the neighbor IDs of the deleted vectors into memory. Loading all of them can cause a high memory consumption (e.g., 512B for a vector with 128 neighbors). OdinANN uses a simple approach, namely loading part of the neighbors for each vector, to meet the memory budget at the cost of index quality.

In the second pass, OdinANN scans the whole index to do three things in a streaming manner. (1) Read a batch of records and replace the deleted vector IDs with their neighbor IDs in memory. (2) Prune neighbors for records whose out-degree is larger than the limit. (3) Write them back to disk.

Updates during merge. Deletes can be trivially handled in memory, so we mainly discuss inserts here. We propose two ways for inserts during merge. First, they can be absorbed by an in-memory index and digested using direct insert after the merge. Second, they can be executed with direct insert by not allocating records in the merging pages. As merge is typically short in OdinANN (§4.7), the overheads of the in-memory index and extra disk space are acceptable.

3.5 Discussion

Consistency. OdinANN uses snapshots and journaling for consistency. Merging creates an index snapshot. Between two merging, journaling is used to record the delta updates for the in-memory and on-disk data structures. For less overhead, journaling may be flushed to disk later than on-disk inserts. To maintain a consistent prefix for updates [28], we roll back the index by (buffered) deleting all the records with newer IDs than the journal. We store the metadata for recovery (e.g., ID and transaction ID) together with each record on disk.

GC-free. OdinANN supports *GC-free inserts*. Compared to the log-structured layout, OdinANN directly reuses the invalid records, eliminating the need for *logical* garbage collection – a process to gather valid records into a new index. Our designs target FTL-based SSDs and do not interfere with their *physical* garbage collection. For *deletes*, to recycle the space of deleted vectors, OdinANN needs the lightweight merge (§3.4.2), which is similar to a logical GC.

Insert latency. Compared to buffered insert in DiskANN, OdinANN has a higher insert latency as it directly writes the updated records to disk. This leads to a ~11.1ms insert latency (§4.4). We believe this latency is acceptable, as real-world systems [7, 22] target indexing latencies around 10ms.

Memory usage. OdinANN stores three things in memory:

- *PQ table* for PQ-compressed vectors, 32 bytes per vector.
- *ID-to-location and ID-to-tag hash tables* contain the mapping of vector ID to 4-byte record location and 4-byte tag (the same as DiskANN [23]), 16 bytes per vector.
- *Location-to-ID hash table* contains the mapping of record location to vector ID, used by the first pass of a merge. We use per-page fixed-size arrays for this, requiring 4 bytes per record (including empty ones) and 4 bytes per page.

For a typical OdinANN index with 2× disk space consumption and 6 records per page, the memory usage is ~58 bytes per vector or 58GB for billion-scale datasets. The memory usage in our evaluation is higher (83.8GB, §4.3). This is due to the memory overheads of hash tables, and free slots caused by dynamic doubling of the PQ table and hash tables.

4 Evaluation

We evaluate OdinANN to answer the following questions:

- How does OdinANN compare to other updatable ANNS indexes for concurrent inserts and searches? (§4.2)
- How does OdinANN perform in billion-scale datasets compared to other ANNS indexes? (§4.3)
- How do the techniques in OdinANN contribute to the performance of direct insert? (§4.4)
- How do OdinANN's techniques for inserts impact internal metrics, e.g., disk consumption and index quality? (§4.5)
- How does the number of out-neighbors impact OdinANN's insert and search performance? (§4.6)
- How does OdinANN perform for workloads with concurrent inserts, deletes, and searches? (§4.7)

4.1 Experimental Setup

Basic configuration. We use one node for evaluation, which has the following configuration:

- **CPU**: 2× 28-core Intel Xeon Gold 6330 @ 2.00GHz;
- **RAM**: 512GB (16× 32GB DDR4 2933MT/s);
- **SSD**: 1× Samsung PM9A3 3.84TB;
- **OS**: Ubuntu 22.04 LTS with Linux kernel 5.15.0.

Datasets. We use three vector datasets for evaluation:

- SIFT100M [12], a dataset containing 100 million `uint8` vectors with a dimension of 128, and 10,000 query vectors.
- DEEP100M [3], a dataset containing 100 million `float` vectors with a dimension of 96, and 10,000 query vectors.
- SIFT1B [12], a dataset containing one billion `uint8` vectors with a dimension of 128, and 10,000 query vectors.

SIFT100M and DEEP100M are subsets of SIFT1B and DEEP1B with 100 million vectors.

Compared systems. We compare OdinANN with two state-of-the-art ANNS indexes: DiskANN [23] and SPFresh [30].

DiskANN is a graph-based index using buffered inserts. OdinANN is implemented based on DiskANN. Unless otherwise stated, we use the following update configurations for them: Insert candidate pool size l_i is 128. Maximum out-neighbors per record R is 96 for SIFT100M and DEEP100M, and 128 for SIFT1B. The maximal number of vectors to prune in delete C is 384. DiskANN merges updates when the in-memory index size reaches 6% of the on-disk index.

For search configurations, they use a beamwidth equal to 4. The default search candidate list l_s is set to 20 for SIFT100M, and 25 for DEEP100M and SIFT1B. The near-root neighbor cache is disabled. For efficiency, we use `io_uring` [13] in replace of `libaio` as their I/O engine.

SPFresh is a variant of SPANN [5], a cluster-based index that divides vectors into clusters. It supports vector updates by updating the target vector in its nearest clusters. To reduce the imbalance of clusters, SPFresh splits and merges them. We configure SPFresh using the same settings as its open-source code. The maximum cluster size is set to 16KB for SIFT and 48KB for DEEP, for the same number of vectors in a cluster.

We do not compare with other on-disk ANNS systems, such as Starling [27], as they are not designed to support updates.

Metrics. OdinANN is designed for online ANNS scenarios, so we evaluate the search performance, search accuracy, and memory usage during concurrent updates. We use multiple search threads to evaluate search performance, as high search throughput is required in real-world workloads (e.g., 5000 QPS [29]). We gradually add search threads to increase throughput while keeping latency relatively low. For OdinANN and DiskANN, we use 32 search threads. For SPFresh, we use 16 search threads in SIFT and 8 threads in DEEP.

4.2 Overall Performance: Insert-Search

In this evaluation, we build indexes using SIFT100M and DEEP100M datasets and then insert another 100 million vectors into them. Multi-thread searches are concurrently executed to show their interference by inserts and merges.

The evaluated systems take different times in the evaluations. This is because we do not insert vectors during